Corpus analysis: from stream to rule to counterexample

This note traces how the three corpus captures led to rule v2. Every number below is produced by the code in this module and can be reproduced with the commands named at the end. It is deliberately thin on method and heavy on figures, so a reader can check a claim against the bytes.

1. The captures and their streams

The corpus is three captures, all A/B TCP exchanges between the same pair of endpoints. The capture module exports each as a normalized JSON in which every direction of every session is one byte stream, with reassembly holes recorded as diagnostics rather than filled with zeros.

capture	capture_id (prefix)	sessions	stream bytes
corpus_capture_01.pcapng	sha256:1a2f3637124ab	3	3036
corpus_capture_02.pcapng	sha256:4aa281b97de9a	2	520
corpus_capture_defects.pcapng	sha256:ae93a5b223808	1	74

`corpus_capture_defects` carries a `gap` in `s1 A_to_B` (2 missing bytes at offset 19) and an `ambiguity` in `s1 B_to_A` (a conflicting overlap at offset 32, where `2008` was kept and `ffff` rejected). The other two captures are clean.

2. Framing: three candidates against the bytes

The length field sits at offset 2, two bytes, big-endian. What it counts is not written down. Three readings were framed against every stream and kept only if the stream was consumed exactly, with no truncated final message and no impossible command byte.

stream	bytes	payload	payload+length	entire message
c01 s1 A_to_B	300	60/300 clean	1/0 broken	1/0 broken
c01 s1 B_to_A	420	60/420 clean	3/420 broken	1/0 broken
c01 s2 A_to_B	420	60/420 clean	2/420 broken	1/0 broken
c01 s2 B_to_A	420	60/420 clean	2/420 broken	1/0 broken
c01 s3 A_to_B	100	20/100 clean	1/0 broken	1/0 broken
c01 s3 B_to_A	1376	20/1376 clean	3/1376 broken	2/1376 broken
c02 s1 A_to_B	100	20/100 clean	1/0 broken	1/0 broken
c02 s1 B_to_A	140	20/140 clean	2/140 broken	1/0 broken
c02 s2 A_to_B	140	20/140 clean	2/140 broken	1/0 broken
c02 s2 B_to_A	140	20/140 clean	2/140 broken	1/0 broken
defects s1 A_to_B	24	4/24 broken	1/0 broken	1/0 broken
defects s1 B_to_A	50	5/50 clean	2/50 broken	1/0 broken

`payload` frames all eleven clean streams exactly and is the only candidate that does. `payload_and_length_field` splits the stream into a few oversized blocks (it consumes the stream but leaves the command bytes at impossible values), and `entire_message` stops after the first size. The twelfth stream, the one with a gap, is the applicability boundary: no framing can frame across a hole, so the message that straddles it is `incomplete`, not `matched`. See `docs/REFERENCE_INVESTIGATION.md` for the full table.

The count here is eleven of twelve because this census covers all three captures. The framing generator (`scripts/run_reference_investigation.py`) runs over its default corpus of the two clean captures, where the same verdict reads as ten of ten; the two numbers differ only in what they count, not in the outcome.

3. The field layout that survived

offset	size	type	name	meaning
0	1	uint8	command	request code
1	1	uint8	flags	reserved in every observed message
2	2	uint16 BE	payload_length	bytes after the length field
4	1	uint8	target	parameter identifier
5	2	uint16 BE	value	written value, present only on writes

The layout is identical in both directions on the wire; the direction only changes which `command` values are legal. Three of the byte meanings are hypotheses (`command`, `target`, `value`), marked `hypothesis: true` in the rule, and the alternatives engine is run on all three.

4. Rule v1 and the eighty counterexamples

Rule v1 scoped to `A_to_B` and declared `command` as the single value `0x01`.

check	matched	mismatched	incomplete	not_applicable
v1 on corpus_capture_01	60	80	0	140
v1 on corpus_capture_02	20	20	0	40
v1 on synthetic_live	0	0	1	1

The eighty mismatches in c01 are all at offset 0, in sessions `s2` (command `0x02`) and `s3` (command `0x03`). Each is bound to its bytes, for example `02000003130055` (`s2`, first counterexample), so a reader can see the byte that broke the rule. The distribution is clean and per-session:

session	command bytes seen	messages
s1	01	60
s2	02	60
s3	03	20

`not_applicable` counts the `B_to_A` streams: the scope says `A_to_B`, so the rule does not claim anything about the other direction, and the engine records that rather than reporting a spurious match.

5. Alternatives: is `0x01` the whole truth?

Before widening the rule, the field was put to the alternatives engine. For `command` in c01 it reports:

reading	support	contradict	score
entropy_enum	140	0	1.00
low_cardinality	140	0	1.00
offset_shift_+1	140	0	1.00
counter	137	2	0.99
constant	60	80	0.43

`constant` (the v1 reading) is contradicted by more than half the messages; `entropy_enum` fits all of them and agrees that the value space is small and closed. That is the evidence behind widening the rule, and it is why step 5 came before step 6.

6. Rule v2 and version comparison

Rule v2 keeps the layout and the framing untouched and widens `command` to `[1, 2, 3]`. Comparing the two reports over the same capture:

transition	resolved	introduced
v1 -> v2 on corpus_capture_01	80	0

check	matched	mismatched	incomplete	not_applicable
v2 on corpus_capture_01	140	0	0	140
v2 on corpus_capture_02	40	0	0	40
v2 on synthetic_live	0	0	1	1

All eighty counterexamples are resolved by the single-shape change, and none is introduced. The old v1 result is marked `outdated`; it is never shown as the current verdict.

7. Where the rule stops

Widening `command` to the corpus values does not make the rule general. On `synthetic_live` the same rule matches nothing. The capture uses a little-endian length, so under the rule's big-endian length both streams are truncated: neither frames cleanly, and the scoped rule reports one `incomplete` message on `A_to_B` with the rest `not_applicable`. The rule fails on the length field before any command byte is judged, and it says so rather than guessing. This is the applicability boundary the whole exercise is built to expose: a rule is confirmed inside the bytes it was checked against, not proven beyond them.

8. The six statuses, each with a witness

The classification is not a list of words in a document; every status is produced by the engine from the shipped captures. `tests/integration/test_status_coverage.py` fails if any of them stops being reachable.

status	witness on the corpus	rule and stream
`matched`	60 requests whose command is in the declared set	rule v1, `corpus_capture_01` `s1` `A_to_B`
`mismatched`	80 requests carrying a command outside `{1}`	rule v1, `corpus_capture_01` `s2 + s3` `A_to_B`
`incomplete`	a message whose framing reaches into a gap	unscoped rule on `corpus_capture_defects`, or a field past the message end
`ambiguous`	the message whose field range covers the conflicting overlap	unscoped rule on `corpus_capture_defects` `B_to_A` (offset 32)
`uncovered`	a message whose only declared field is an optional field that is absent	optional trailing field on `corpus_capture_01`
`not_applicable`	every response the scoped rule does not interpret	rule v1, `corpus_capture_01` `B_to_A`

`outdated` and `unknown` sit outside the core six: `outdated` is a versioning mark on a superseded result (section 6), not a reading of bytes, and `unknown` is the engine's default before any rule is applied.

9. How to reproduce

```
python -m pytest tests/integration/test_reference_report.py -q
python -m src.protocol.cli verify --rule examples/corpus_rule_v1.json \
    --capture tests/corpus/reference_export/corpus_capture_01.normalized.json \
    --out /tmp/v1.json
python -m src.protocol.cli alternatives --rule examples/corpus_rule_v1.json \
    --report /tmp/result.json --out /tmp/alternatives.json
```

The framing census is produced by `scripts/run_reference_investigation.py`.

10. The organizer dataset

This analysis uses the synthetic corpus, because it exercises the framing choices deliberately. The organizer supplied a second, independent capture set for the thermal-stand protocol; it is committed under `tests/corpus/real/` and studied in `docs/REAL_CORPUS_ANALYSIS.md`. It runs the same loop against a different protocol (`0xa65c` magic, a header-counting length field): the naive rule leaves six counterexamples, the header-bias rule clears them, and all 90 messages across four captures match with full byte coverage. The synthetic corpus stays the reference for the framing discussion; the real corpus is the applicability check.

```
python -m scripts.investigate_dataset
python -m pytest tests/hypothesis/test_real_corpus.py -q
```